```
# -*- coding: utf-8 -*-
"""Untitled1.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1pH7Z47v9aQLKiFdJdavsvaVfdcrG5g_a
"""



"""# Task
Load the 'Global_Weather_Repository.csv' dataset, perform comprehensive data cleaning and preprocessing including handling missing values and inconsistencies, converti

## Setup Environment and Load Data
"""

import pandas as pd

df = pd.read_csv('master_dataset.csv')
df.head(5)

print(df.shape)
df.info()

"""## Inspect Dataset Structure and Data Types



"""

df.info()
df.head()
df.describe()

"""## Identify Missing Values and Anomalies



"""

df.isnull().sum()

anomalous_columns = [
    'air_quality_Carbon_Monoxide',
    'air_quality_Sulphur_dioxide',
    'air_quality_PM10',
    'wind_mph',
    'wind_kph',
    'gust_mph',
    'gust_kph',
    'pressure_mb',
    'pressure_in'
]

for col in anomalous_columns:
    print(f"{col}: Min = {df[col].min()}, Max = {df[col].max()}")

print(f"'air_quality_Carbon_Monoxide' with -9999: {(df['air_quality_Carbon_Monoxide'] == -9999).sum()} occurrences")
print(f"'air_quality_Sulphur_dioxide' with -9999: {(df['air_quality_Sulphur_dioxide'] == -9999).sum()} occurrences")
print(f"'air_quality_PM10' with -1848.15: {(df['air_quality_PM10'] == -1848.15).sum()} occurrences")

import numpy as np

df.loc[df['air_quality_Carbon_Monoxide'] == -9999, 'air_quality_Carbon_Monoxide'] = np.nan
df.loc[df['air_quality_Sulphur_dioxide'] == -9999, 'air_quality_Sulphur_dioxide'] = np.nan
df.loc[df['air_quality_PM10'] == -1848.15, 'air_quality_PM10'] = np.nan

anomalous_columns = [
    'air_quality_Carbon_Monoxide',
    'air_quality_Sulphur_dioxide',
    'air_quality_PM10',
    'wind_mph',
    'wind_kph',
    'gust_mph',
    'gust_kph',
    'pressure_mb',
    'pressure_in'
]
for col in anomalous_columns:
    print(f"{col}: Min = {df[col].min()}, Max = {df[col].max()}")

df.isnull().sum()

print(df[['air_quality_Carbon_Monoxide', 'air_quality_Sulphur_dioxide', 'air_quality_PM10']].dtypes)

nan_rows_carbon_monoxide = df[df['air_quality_Carbon_Monoxide'].isna()]
nan_rows_sulphur_dioxide = df[df['air_quality_Sulphur_dioxide'].isna()]
nan_rows_pm10 = df[df['air_quality_PM10'].isna()]

if not nan_rows_carbon_monoxide.empty:
    print(nan_rows_carbon_monoxide[['country', 'location_name', 'air_quality_Carbon_Monoxide']])
else:
    print("No NaN values found in 'air_quality_Carbon_Monoxide' despite replacement.")

if not nan_rows_sulphur_dioxide.empty:
    print(nan_rows_sulphur_dioxide[['country', 'location_name', 'air_quality_Sulphur_dioxide']])
else:
    print("No NaN values found in 'air_quality_Sulphur_dioxide' despite replacement.")

if not nan_rows_pm10.empty:
    print(nan_rows_pm10[['country', 'location_name', 'air_quality_PM10']])
else:
    print("No NaN values found in 'air_quality_PM10' despite replacement.")

affected_columns = [
    'air_quality_Carbon_Monoxide',
    'air_quality_Sulphur_dioxide',
    'air_quality_PM10'
]

for col in affected_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df.isnull().sum()

"""## Handle Missing and Inconsistent Entries



"""

import numpy as np
```

```python
or_value = df.loc[0, 'air_quality_Carbon_Monoxide']
df.loc[0, 'air_quality_Carbon_Monoxide'] = np.nan

print(f"Value at df.loc[0, 'air_quality_Carbon_Monoxide'] set to: {df.loc[0, 'air_quality_Carbon_Monoxide']}")
print(f"'air_quality_Carbon_Monoxide' missing values count: {df['air_quality_Carbon_Monoxide'].isnull().sum()}")

df.loc[0, 'air_quality_Carbon_Monoxide'] = or_value
print(f"Value at df.loc[0, 'air_quality_Carbon_Monoxide'] reverted to: {df.loc[0, 'air_quality_Carbon_Monoxide']}")

import numpy as np

df.loc[df['air_quality_Carbon_Monoxide'] == -9999, 'air_quality_Carbon_Monoxide'] = np.nan
df.loc[df['air_quality_Sulphur_dioxide'] == -9999, 'air_quality_Sulphur_dioxide'] = np.nan
df.loc[df['air_quality_PM10'] == -1848.15, 'air_quality_PM10'] = np.nan

df[['air_quality_Carbon_Monoxide', 'air_quality_Sulphur_dioxide', 'air_quality_PM10']].isnull().sum()

import numpy as np

wind_mph_threshold = 200
wind_kph_threshold = 320

pressure_mb_lower_threshold = 850
pressure_mb_upper_threshold = 1080
pressure_in_lower_threshold = 850 / 33.8639
pressure_in_upper_threshold = 1080 / 33.8639

df.loc[df['wind_mph'] > wind_mph_threshold, 'wind_mph'] = np.nan
df.loc[df['wind_kph'] > wind_kph_threshold, 'wind_kph'] = np.nan
df.loc[df['gust_mph'] > wind_mph_threshold, 'gust_mph'] = np.nan
df.loc[df['gust_kph'] > wind_kph_threshold, 'gust_kph'] = np.nan

df.loc[(df['pressure_mb'] < pressure_mb_lower_threshold) | (df['pressure_mb'] > pressure_mb_upper_threshold), 'pressure_mb'] = np.nan
df.loc[(df['pressure_in'] < pressure_in_lower_threshold) | (df['pressure_in'] > pressure_in_upper_threshold), 'pressure_in'] = np.nan

df[['wind_mph', 'wind_kph', 'gust_mph', 'gust_kph', 'pressure_mb', 'pressure_in']].isnull().sum()

imputation_columns = [
    'air_quality_Carbon_Monoxide',
    'air_quality_Sulphur_dioxide',
    'air_quality_PM10',
    'wind_mph',
    'wind_kph',
    'gust_mph',
    'gust_kph',
    'pressure_mb',
    'pressure_in'
]

print("Missing Values Before Imputation")
print(df[imputation_columns].isnull().sum())

for col in imputation_columns:
    median_val = df[col].median()
    df[col].fillna(median_val, inplace=True)

print("Missing Values After Imputation")
print(df[imputation_columns].isnull().sum())

imputation_columns = [
    'air_quality_Carbon_Monoxide',
    'air_quality_Sulphur_dioxide',
    'air_quality_PM10',
    'wind_mph',
    'wind_kph',
    'gust_mph',
    'gust_kph',
    'pressure_mb',
    'pressure_in'
]

print("Missing Values Before Imputation:")
print(df[imputation_columns].isnull().sum())

for col in imputation_columns:
    median_val = df[col].median()
    df[col] = df[col].fillna(median_val)

print("Missing Values After Imputation:")
print(df[imputation_columns].isnull().sum())

"""###Convert units and normalize values."""

datetime_columns = [
    'last_updated',
    'sunrise',
    'sunset',
    'moonrise',
    'moonset'
]

print("Data Types Before Datetime Conversion:")
print(df[datetime_columns].dtypes)

df['last_updated'] = pd.to_datetime(df['last_updated'], format='%Y-%m-%d %H:%M', errors='coerce')
for col in ['sunrise', 'sunset', 'moonrise', 'moonset']:
    df[col] = pd.to_datetime(df[col], format='%I:%M %p', errors='coerce')

print("Data Types After Datetime Conversion:")
print(df[datetime_columns].dtypes)

display(df[datetime_columns].head())

print("Data Type Before Categorical Conversion:")
print(df['moon_phase'].dtype)

df['moon_phase'] = df['moon_phase'].astype('category')

print("Data Type After Categorical Conversion:")
print(df['moon_phase'].dtype)

display(df['moon_phase'].head())

imperial_columns_to_drop = [
    'temperature_fahrenheit',
    'wind_mph',
    'pressure_in',
    'precip_in',
    'visibility_miles',
    'gust_mph'
]
```

```
df = df.drop(columns=imperial_columns_to_drop)

df.head()

from sklearn.preprocessing import MinMaxScaler

columns_to_normalize = [
    'temperature_celsius',
    'wind_kph',
    'pressure_mb',
    'precip_mm',
    'humidity',
    'cloud',
    'visibility_km',
    'uv_index',
    'gust_kph',
    'air_quality_Carbon_Monoxide',
    'air_quality_Ozone',
    'air_quality_Nitrogen_dioxide',
    'air_quality_Sulphur_dioxide',
    'air_quality_PM2.5',
    'air_quality_PM10'
]

scaler = MinMaxScaler()

df[columns_to_normalize] = scaler.fit_transform(df[columns_to_normalize])

display(df[columns_to_normalize].head())
display(df[columns_to_normalize].describe())

"""## Aggregate or Filter Data
Aggregate the data by computing monthly averages for temperature to prepare it for further analysis.

"""

df['month'] = df['last_updated'].dt.month

display(df[['last_updated', 'month']].head())

monthly_avg_temp = df.groupby(['country', 'month'])['temperature_celsius'].mean().reset_index()

display(monthly_avg_temp.head())

"""### Data Cleaning and Preprocessing Summary Report

This report details the data cleaning and preprocessing steps applied to the 'Global_Weather_Repository.csv' dataset, aiming to prepare it for further analysis.

#### **Data Loading and Initial Inspection**
The dataset 'master_dataset.csv' was loaded into a pandas DataFrame named `df`. Initial inspection involved using `df.info()` to understand column data types and non-

#### **Missing Value and Anomaly Handling**
Initially, `df.isnull().sum()` reported no missing values, but `df.describe()` and manual checks revealed sentinel values like -9999 and -1848.15 in air quality columns

Furthermore, extreme outliers in wind-related columns (`wind_mph`, `wind_kph`, `gust_mph`, `gust_kph`) exceeding 200 mph (approx 320 kph) and pressure columns (`pressu

Finally, all introduced `np.nan` values across the air quality, wind, and pressure columns were imputed using the median value of their respective columns. This strate

#### **Data Type Conversions**
Several columns underwent data type conversions to ensure proper handling and analysis:
*   The 'last_updated' column was converted to datetime objects using `pd.to_datetime` with the format '%Y-%m-%d %H:%M'.
*   'sunrise', 'sunset', 'moonrise', and 'moonset' columns, initially object type, were converted to datetime objects using `pd.to_datetime` with the format '%I:%M %p
*   The 'moon_phase' column was converted to a categorical data type (`category`) to optimize memory usage and accurately represent its discrete nature.

#### **Unit Standardization**
To standardize the dataset to metric units and remove redundant columns, the following imperial unit columns were dropped from the DataFrame:
*   `temperature_fahrenheit`
*   `wind_mph`
*   `pressure_in`
*   `precip_in`
*   `visibility_miles`
*   `gust_mph`

#### **Value Normalization**
Min-Max scaling was applied to several key numerical features to transform their values into a common range between 0 and 1. This normalization is crucial for algorith
`temperature_celsius`, `wind_kph`, `pressure_mb`, `precip_mm`, `humidity`, `cloud`, `visibility_km`, `uv_index`, `gust_kph`, `air_quality_Carbon_Monoxide`, `air_qualit

#### **Data Aggregation**
For further analysis, a new 'month' column was extracted from the 'last_updated' datetime column. The data was then aggregated by 'country' and 'month' to calculate th

This comprehensive cleaning and preprocessing ensures the dataset is now consistent, free from critical anomalies, standardized, and ready for robust statistical analy

## Save the Cleaned Data

### Subtask:
Save the cleaned and processed DataFrame to a new CSV file.

**Reasoning**:
As per instruction 1 and 2, I will save the cleaned and processed `df` DataFrame to a new CSV file named 'cleaned_global_weather_data.csv', ensuring that the DataFrame
"""

df.to_csv('cleaned_global_weather_data.csv', index=False)

"""## Final Task

### Subtask:
Conclude the data preparation, indicating that the dataset is now clean and ready for deeper analysis.

## Summary:

### Q&A
The data cleaning and preprocessing steps undertaken for the 'Global_Weather_Repository.csv' dataset involved initial inspection, robust handling of missing values and

### Data Analysis Key Findings
*   **Initial Data Inspection:** The dataset `master_dataset.csv` was loaded and inspected using `df.info()`, `df.head()`, and `df.describe()` to understand its struct
*   **Missing Value and Anomaly Handling:**
    *   Sentinel values like -9999 and -1848.15 were identified in air quality columns (`air_quality_Carbon_Monoxide`, `air_quality_Sulphur_dioxide`, `air_quality_PM1
    *   Extreme outliers in wind-related columns (exceeding \$200 \text{ mph}) and pressure columns (outside \$850-1080 \text{ mb}) were also converted to `np.nan`.
    *   All introduced `np.nan` values across these affected columns were imputed using their respective column medians.
*   **Data Type Conversions:**
    *   `last_updated` was converted to datetime objects.
    *   `sunrise`, `sunset`, `moonrise`, and `moonset` were converted to datetime objects.
    *   `moon_phase` was converted to a categorical data type for efficiency.
*   **Unit Standardization:** Six imperial unit columns (`temperature_fahrenheit`, `wind_mph`, `pressure_in`, `precip_in`, `visibility_miles`, `gust_mph`) were droppe
*   **Value Normalization:** Min-Max scaling was applied to 15 key numerical features (including `temperature_celsius`, `wind_kph`, `pressure_mb`, and various air qual
*   **Data Aggregation:** A new 'month' column was extracted, and the data was aggregated by 'country' and 'month' to calculate the monthly average of `temperature_cel
*   **Cleaned Data Export:** The processed DataFrame was successfully saved as `'cleaned_global_weather_data.csv'` without the DataFrame index.

### Insights or Next Steps
*   The dataset is now meticulously cleaned, consistent, standardized, and free from critical anomalies, making it suitable for robust statistical analysis and machine
*   The cleaned data can now be used for deeper exploratory data analysis to uncover trends, patterns, and correlations, or proceed directly to model building based on
"""
```